



Recibe una cálida:

# ¡Bienvenida!

---

Te estábamos esperando 😊 +



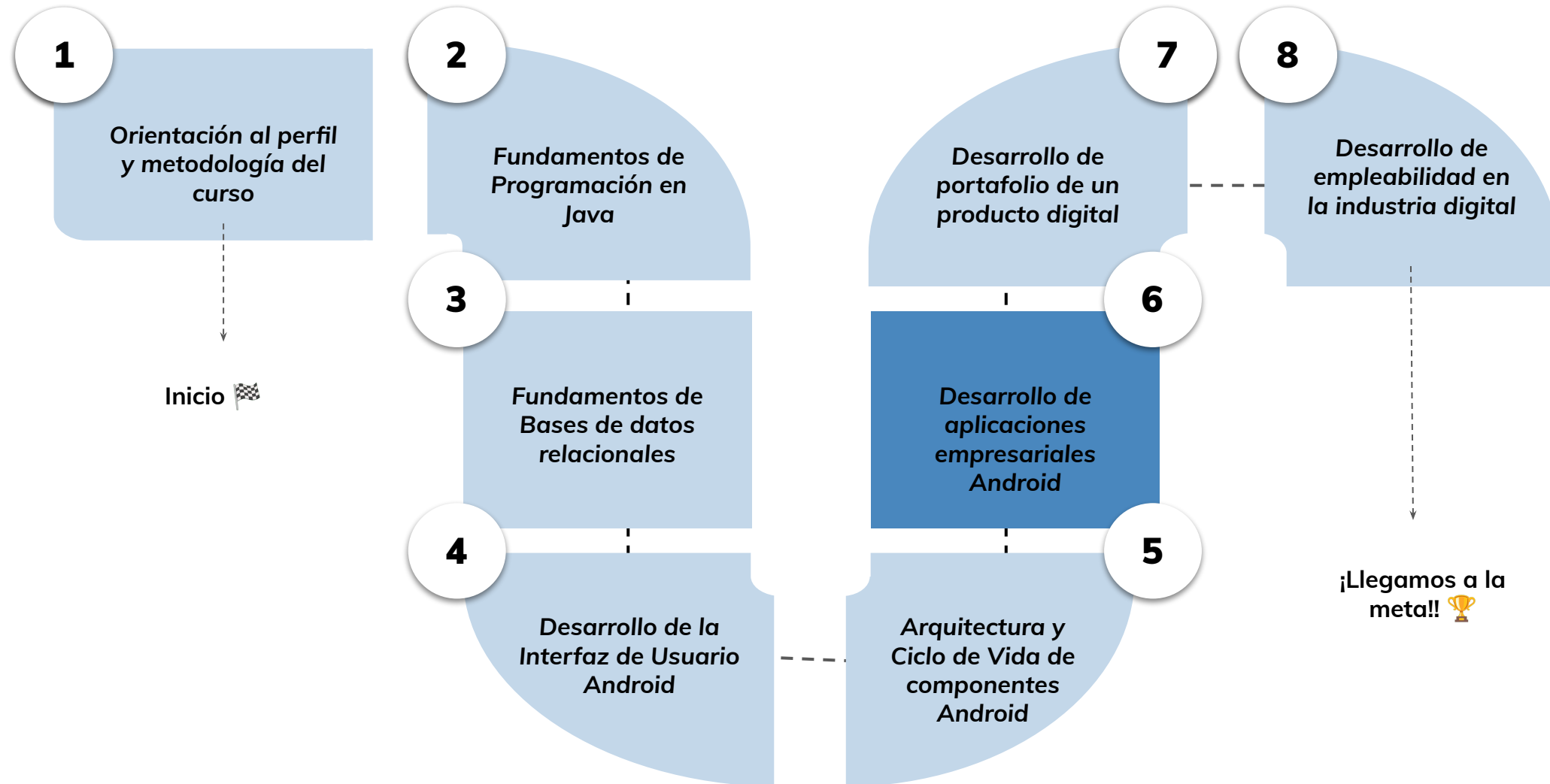
# › Acceso a datos en Android - Parte I

---

**AE1:** Implementar capa de acceso a datos en un aplicativo móvil utilizando la librería room para otorgar persistencia de estados resolviendo el problema planteado.

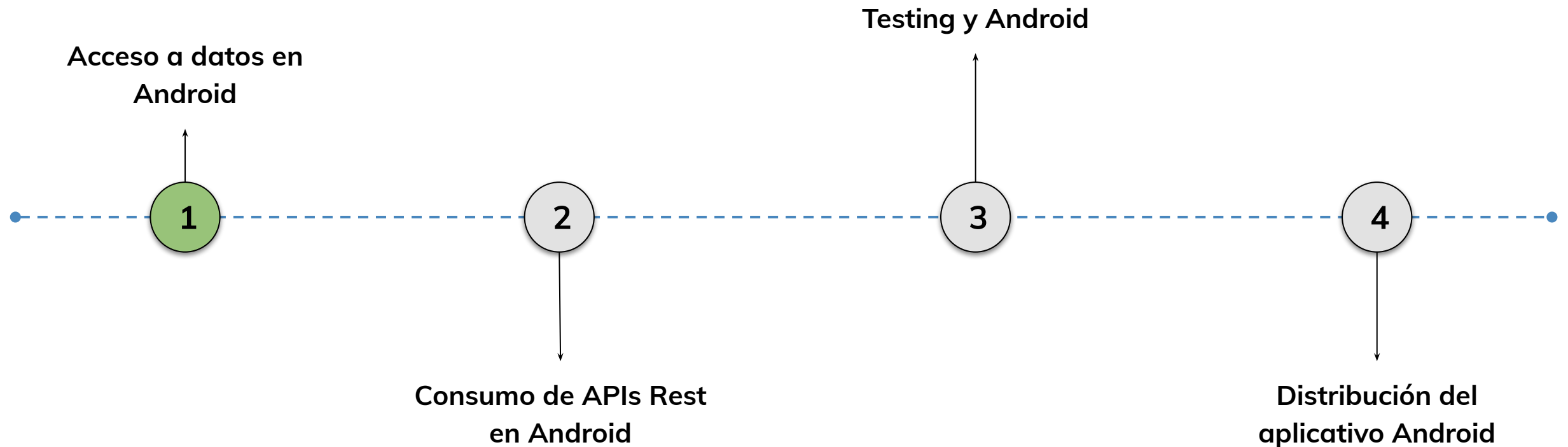
# Hoja de ruta

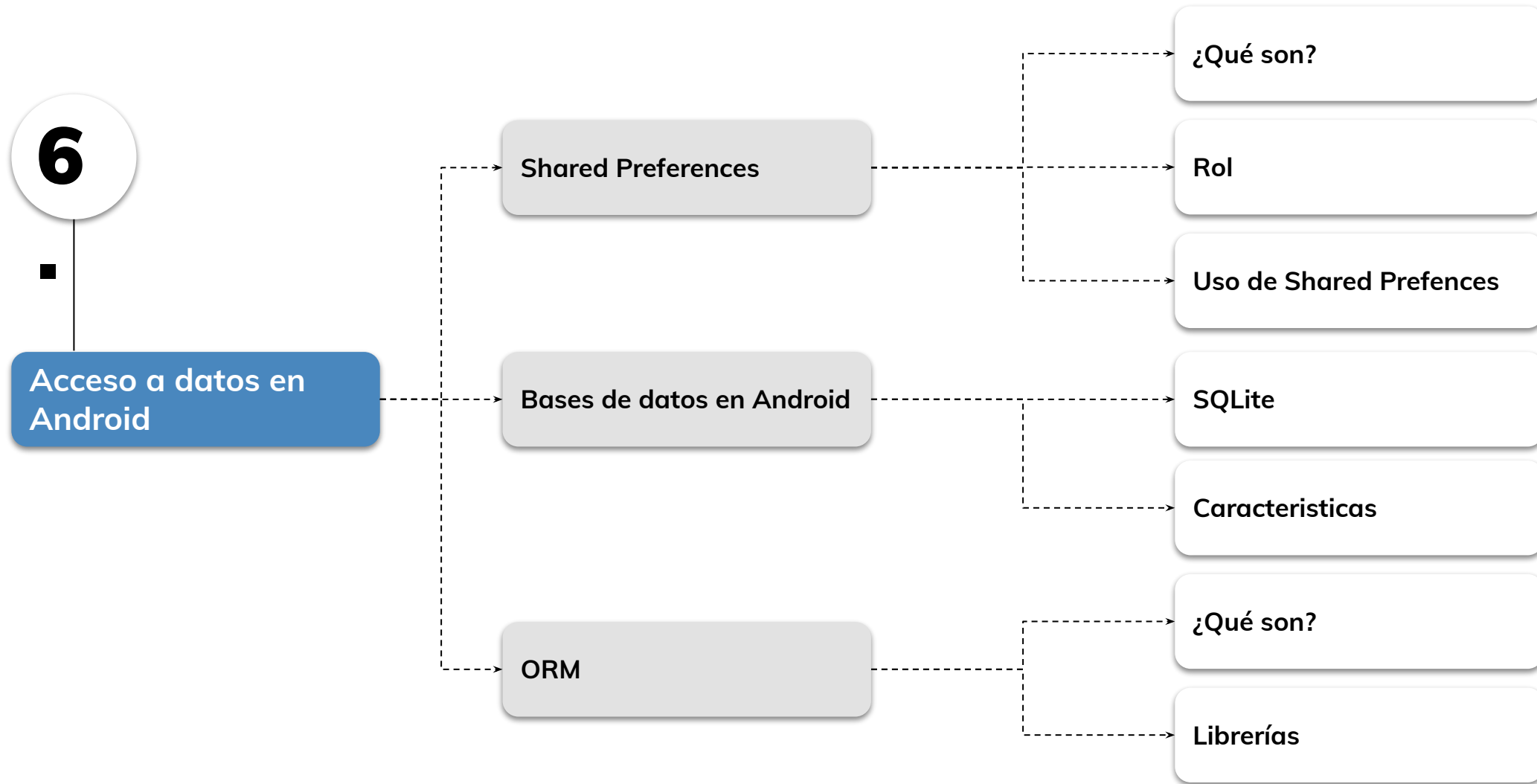
¿Cuáles **módulos** conforman el programa?



# Roadmap de la lección

¿Cuáles lecciones estaremos estudiando en este módulo?







# Objetivos de aprendizaje

¿Qué aprenderás?



- Entender qué son las Shared Preferences y sus ventajas
- Comprender cuál es el rol de SharedPreferences en Android
- Ver cómo aplicar SharedPreferences a un proyecto
- Conocer las características de SQLite y su uso en Android
- Saber cómo funcionan las librerías ORM
- Ver cuáles son las librerías podemos usar para usar bases de datos en Android



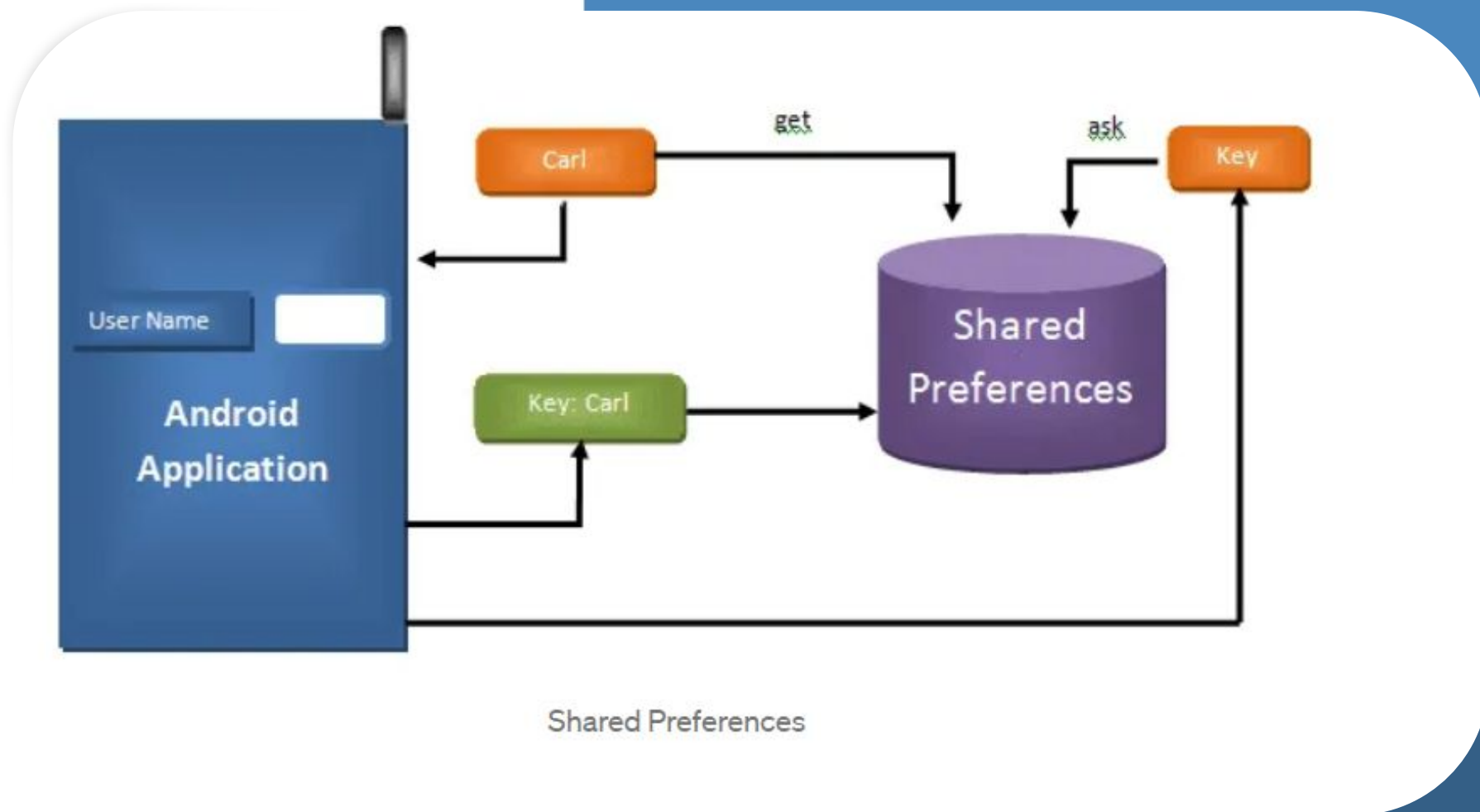
Hay datos que vamos a querer guardar en forma persistente que no requieren ser ordenados en una base de datos o una complicada forma de almacenamiento.

¿Qué datos creen que sería más práctico guardar en un formato simple?

# › Shared Preferences



# SHARED PREFERENCES



# PERSISTENCIA



# Guardar datos en una App

¿Qué ocurre con las apps de redes sociales cuando no tenemos conexión a internet?

- Almacenar datos de nuestros usuarios.
- Parte de las funcionalidades deben guardarse.
- Casos de uso de nuestra app.
- No siempre tendremos acceso a Internet.



# TIPOS DE PERSISTENCIA DE DATOS EN ANDROID

- Interno
- Externo
- Shared Preferences
- Base de datos.



# ALMACENAMIENTO INTERNO

- Almacenamiento privado de archivos.
- Permiso de acceso Aplicación usuarios.
- No expone contenido a otras apps a no ser que lo hagamos explícitamente con un ContentProvider.



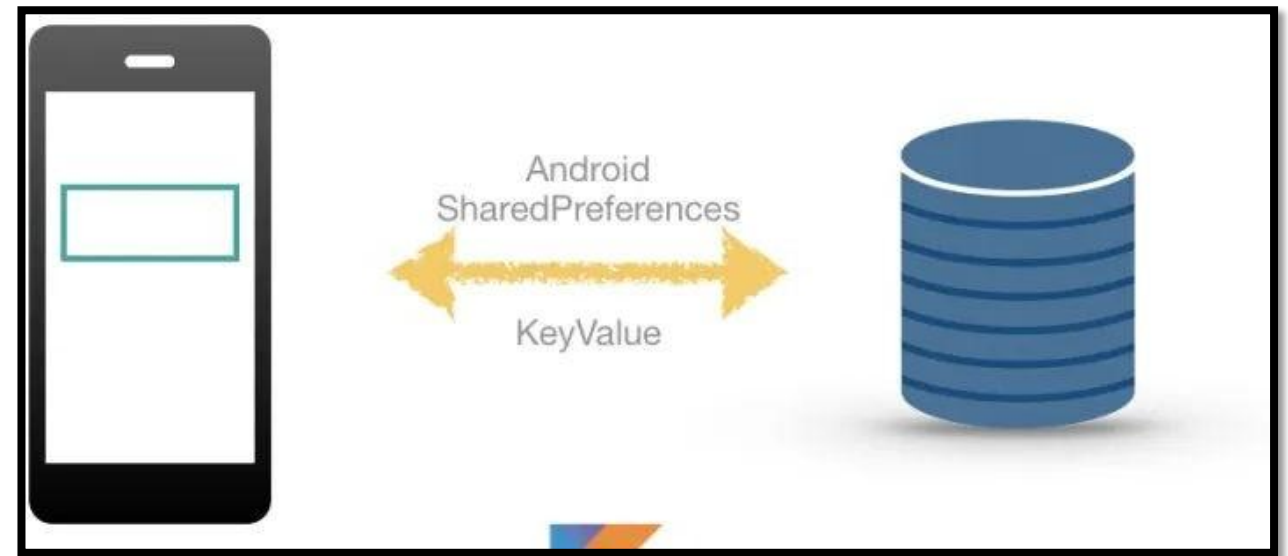
# ALMACENAMIENTO EXTERNO

- Puede ser extraíble
- No se recomienda almacenar información privada.
- Reservado para contenido de gran tamaño
- Se utiliza para guardar archivos que se puedan compartir.



# Shared Preferences

- Almacenamiento de tipos de datos primitivos (int, float, string)
- Formato clave-valor
- Seguridad mínima.
- Pueden ser eliminadas por el usuario.
- Ideal para almacenar preferencias del usuario, datos o información trivial para el funcionamiento de las apps.



# Base de datos SQLite

- Almacenamiento de información estructurada.
- Privado, a no ser que se exponga con un content provider.
- Eficiente para almacenar relaciones entre datos (Tablas)
- Regularmente se utiliza a través de un [ORM](#).

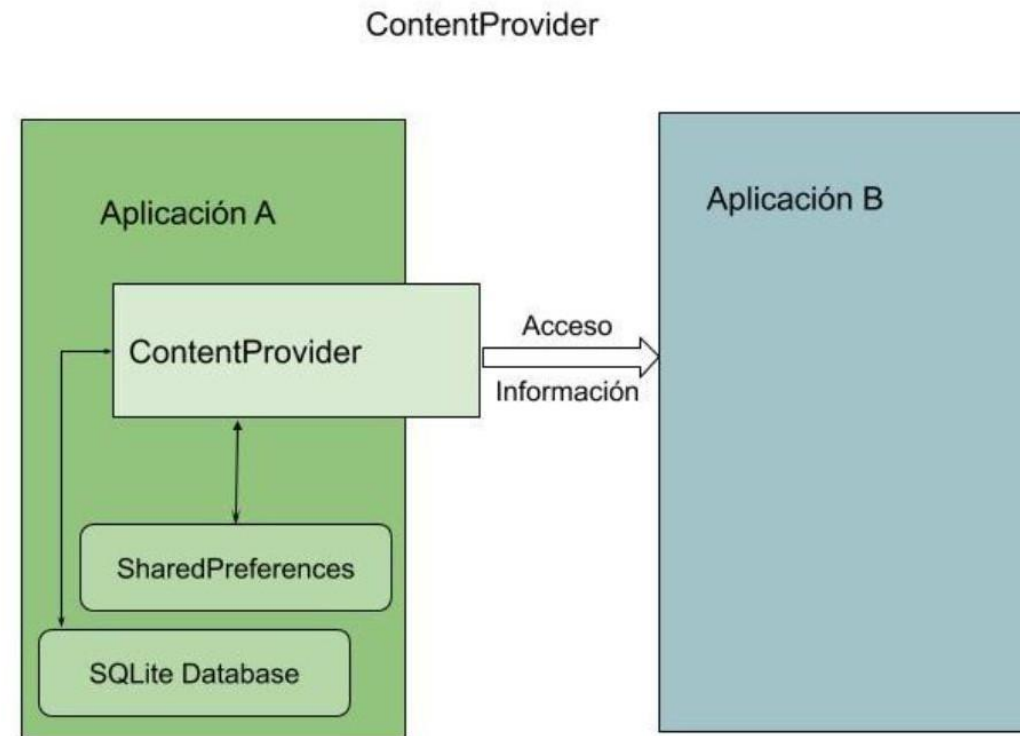




# ¿ContentProvider?

## ContentProvider

Interfaz  
que se expone  
de una aplicación  
a otras para que  
puedan acceder a  
información privada,  
de forma segura  
y confiable



# ¿Que debemos saber acerca de persistencia de datos en android?

- Diferencias entre datos privados y públicos.
- Opciones de persistencia de datos.
- Usar SharedPreferences como método de persistencia.  
En el futuro o sea mañana...
- Utilizar alguna capa de abstracción sobre [SQLite](#).
- Que es un ORM y porque Room Library.



# Resumen y características de persistencia

|                                    | Tipo de contenido                 | Método de acceso                                                                                                                                                                            | Necesidad de permisos                                                                                                                   | Acceso desde otras apps                                                            | Se borran los archivos al desinstalar la app? |
|------------------------------------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------|
| <a href="#">App-specific files</a> | Archivos solo para tu aplicación. | Desde el internal storage, <code>getFilesDir()</code> o <code>getCacheDir()</code><br><br>Desde external storage., <code>getExternalFilesDir()</code> or <code>getExternalCacheDir()</code> | Nunca se necesitan para el almacenamiento interno.<br><br>No son necesarios si nuestra app utiliza Android 4.4 API level 19 o superior. | No.<br><br>Si, si los archivos están dentro de un directorio del external storage. | Si.                                           |

# Resumen y características

|                                             | Tipo de contenido                                      | Método de acceso                            | Necesidad de permisos | Acceso desde otras apps               | Se borran los archivos al desinstalar la app? |
|---------------------------------------------|--------------------------------------------------------|---------------------------------------------|-----------------------|---------------------------------------|-----------------------------------------------|
| <a href="#">Documentos y otros archivos</a> | Cualquier otro tipo de archivo que se pueda compartir. | Storage Access Framework                    | Ninguna               | Si, a través del sistema de archivos. | No                                            |
| <a href="#">App preferences</a>             | Key-value                                              | <a href="#">Jetpack Preferences</a> library | Ninguna               | No                                    | Si                                            |
| Database                                    | Datos estructurados                                    | <a href="#">Room</a> persistence library    | Ninguna               | No                                    | Si                                            |

# ¿Cómo elegir una alternativa de almacenamiento?

Para poder escoger una solución que se adapte a nuestra necesidad podemos realizarnos las siguientes preguntas.

- ¿Cuánto espacio utilizaran los datos?
- ¿Que tan confiable debe ser el acceso a los datos?
- ¿Que tipo de datos necesitas almacenar?
- ¿Los datos serán privados o sensibles en tu App?



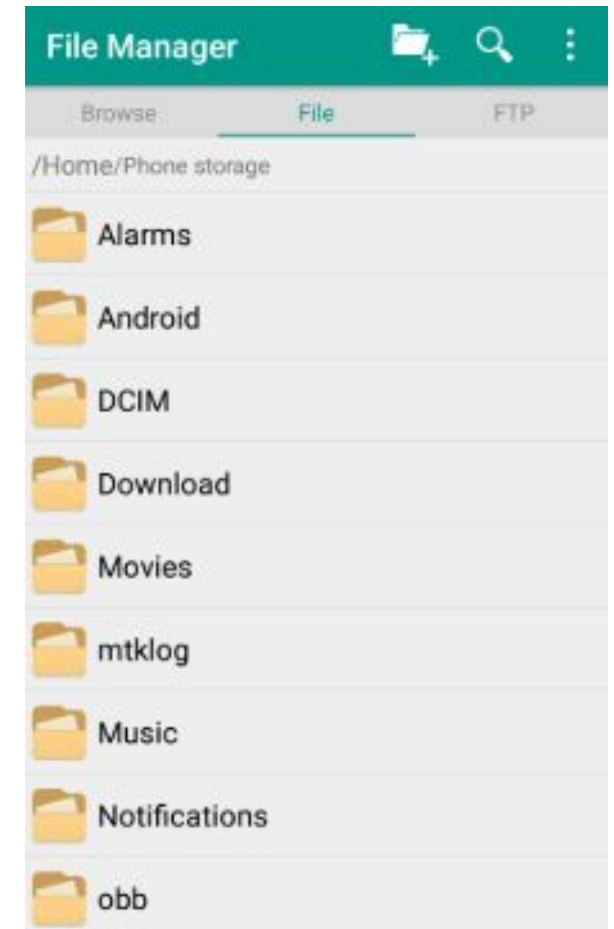
# Categorías de Storage por ubicación

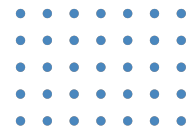
Android nos proporciona dos tipos físicos de ubicación para almacenamiento:

**interno** y **externo** (internal storage & external Storage).

Regularmente en casi todos los dispositivos el almacenamiento interno es más pequeño que el externo, pero el interno está siempre disponible en todos los dispositivos, haciéndolo más confiable para poner información que nuestra app pueda necesitar.

**Alerta:** La ubicación exacta de donde estos archivos serán guardados difiere entre los distintos tipos de dispositivos, por esta razón, nunca usar rutas Hardcodeadas.





# Categorías de Storage por ubicación



Las apps por defecto se instalan en internal storage. Si nuestro **APK** o **Bundle** es de mucho tamaño, podemos indicar una parámetro para que el usuario pueda instalar nuestra app en external storage añadiendo lo siguiente a nuestro Manifest.xml

```
<manifest ...  
    android:installLocation="preferExternal">  
    ...  
</manifest>
```



# Permisos y acceso almacenamiento externo



Para poder acceder a nuestro almacenamiento externo, debemos solicitar los siguientes permisos de escritura y lectura:



[READ\\_EXTERNAL\\_STORAGE](#)



[WRITE\\_EXTERNAL\\_STORAGE](#)

```
<uses-permission
android:name="android.permission.READ_EXTERNAL_S
TORAGE"/>

<uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE
"/>
```



# Scope storage (Lo que viene)

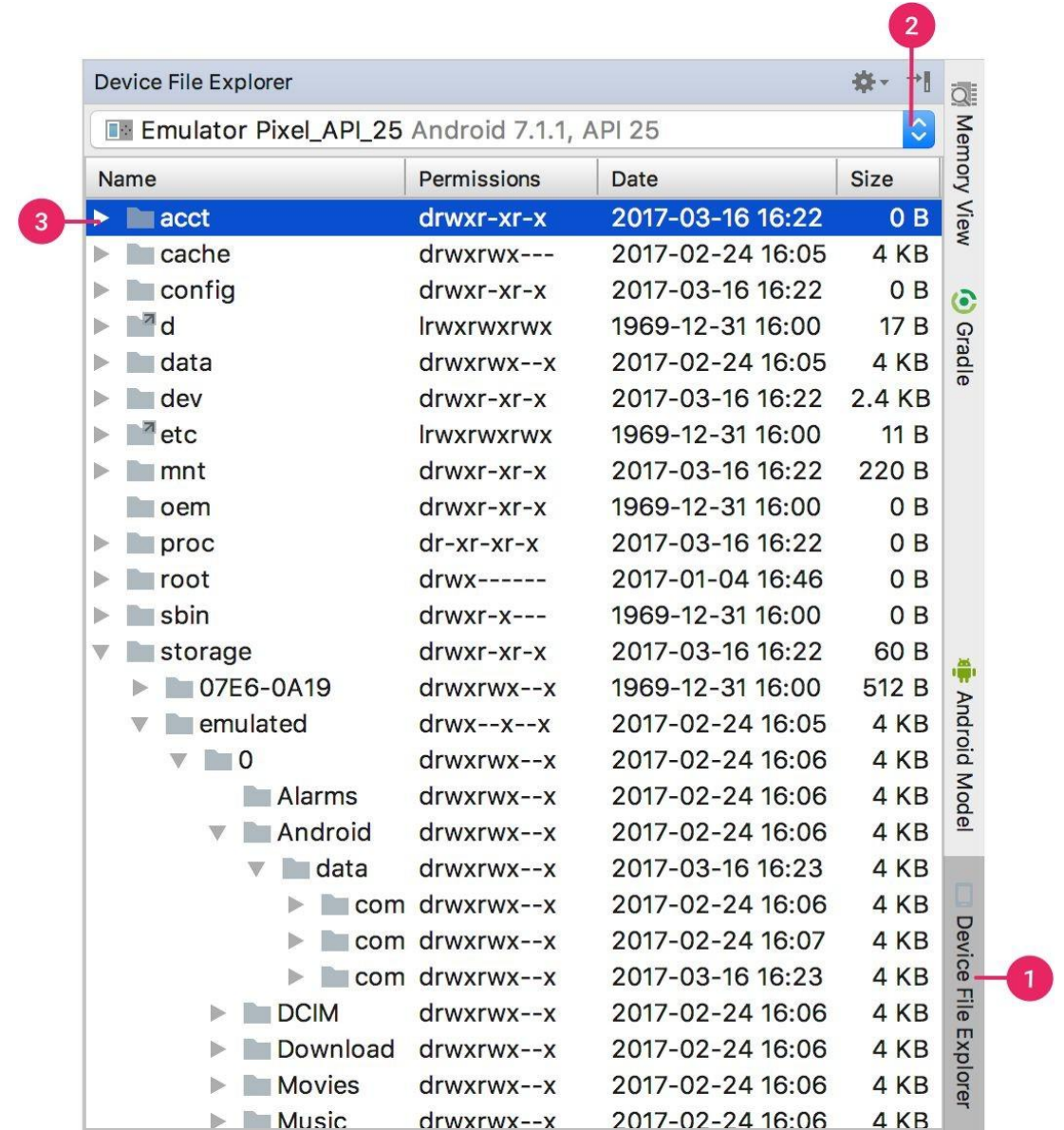
Para darle a los usuarios mayor control sobre los archivos, a partir de la API 29 ( android 10) y superior, se comenzará a utilizar un “scope storage” por defecto. Esto quiere decir que la APP solo tendrá acceso al directorio que lo contiene en external storage y tambien solo a los archivos que la misma app haya creado.Utilizar ese tipo de scope Storage no nos libra de solicitar los permisos para utilizar el external storage.

Esta feature es relativamente nueva, pueden ver más detalles en este [video](#).



# Ver archivos del dispositivo

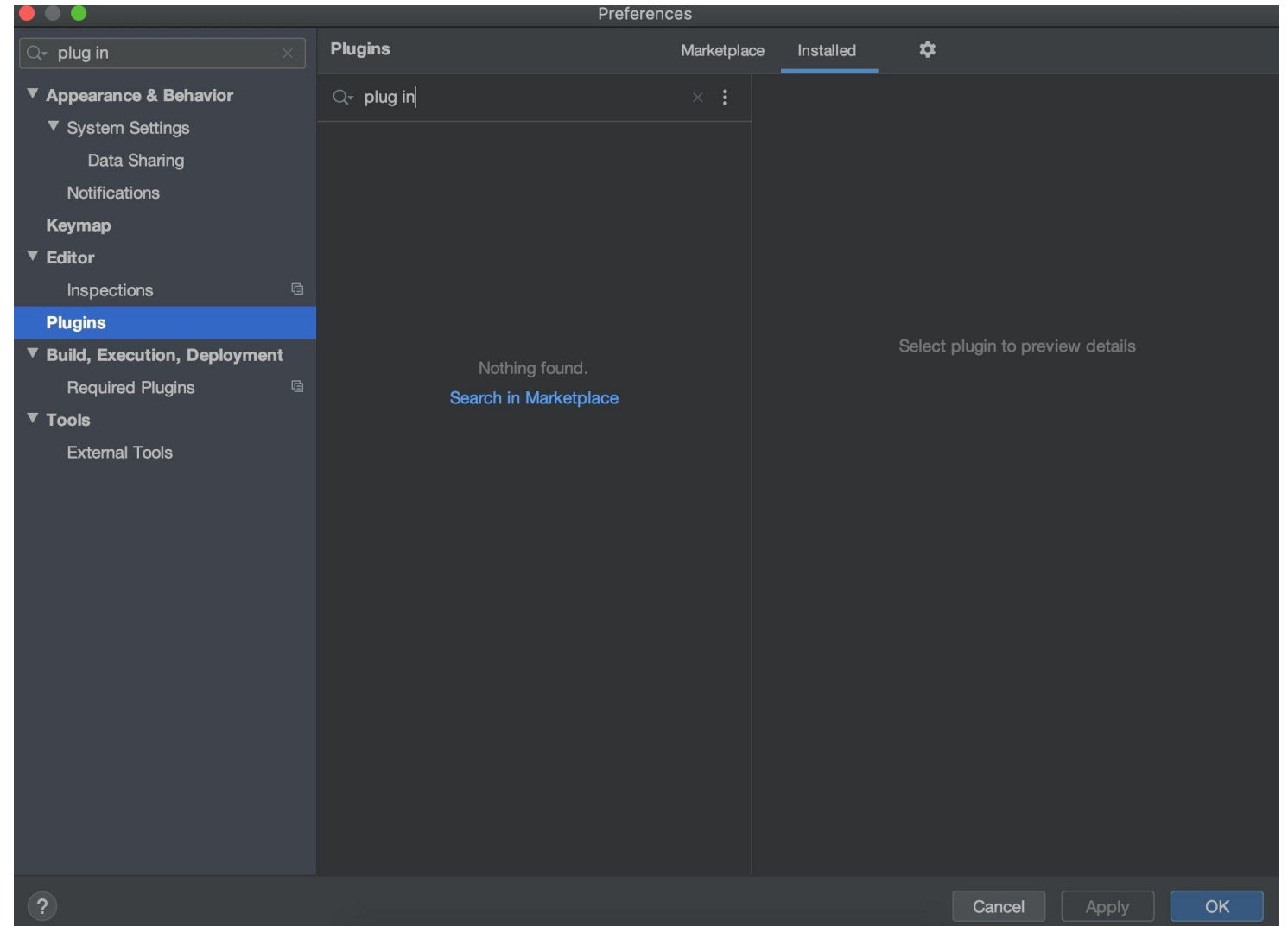
Para ver los archivos que pueda tener el dispositivo, recuerda que puedes utilizar la herramienta [Device File Explorer](#) que está presente en android studio.



# Utilidades (Instalar un Plugin)

Instalar un Plugins en android studio.

Abrir las preferencias y buscar  
Plugins:



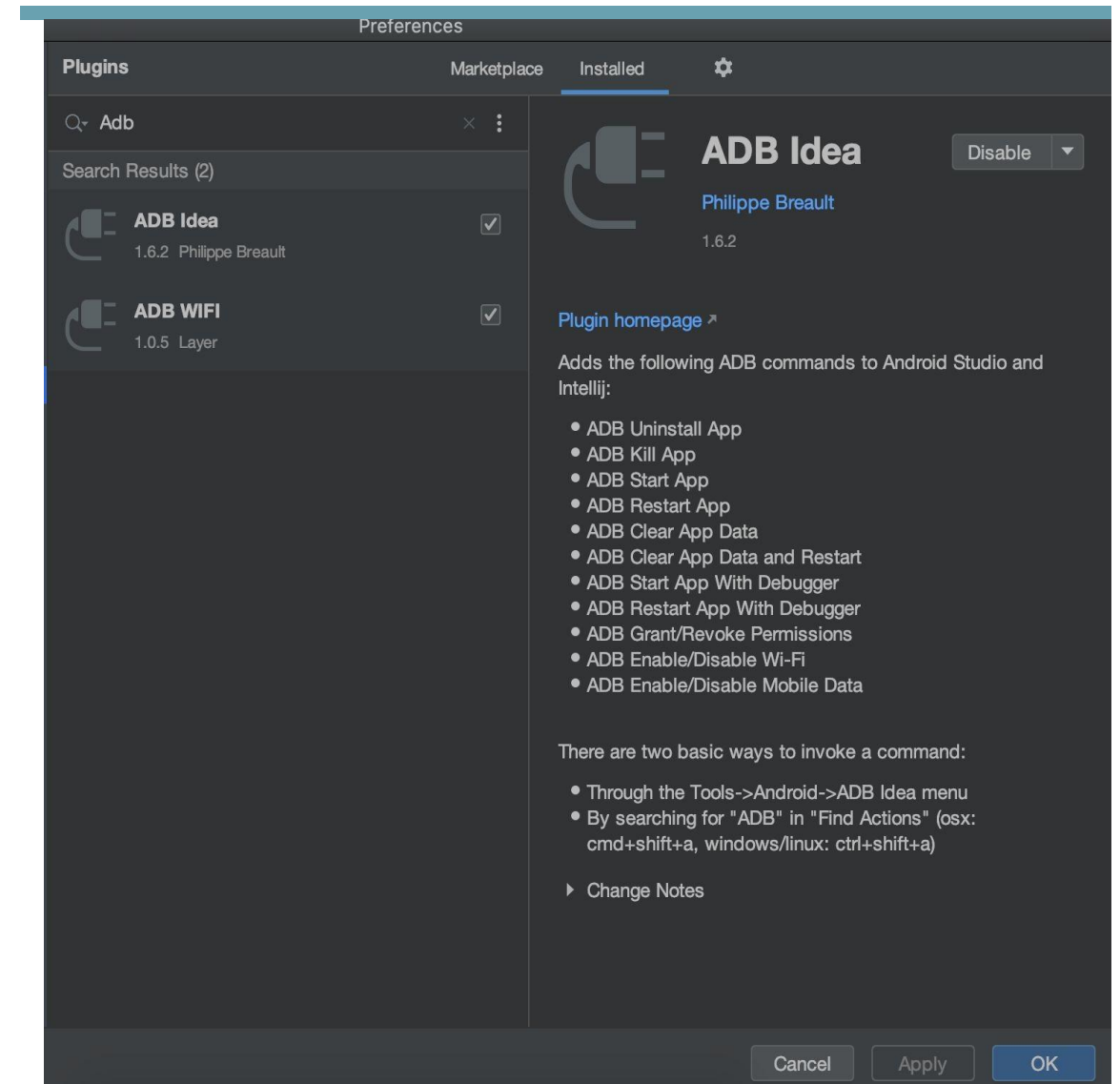
# Instalar ADB idea

Instalaremos el Plugin ADB idea, el cual nos permite realizar algunas operaciones sobre un dispositivo.

Entre sus múltiples funciones, podremos borrar los datos de una App, sin tener que desinstalar del equipo.

Shortcut:

Ctrl + mayús + A



# Database Debug (Opcional)

También podemos incorporar al proyecto la siguiente dependencia, para instalar una utilizada de debug de la base de datos y de SharedPreferences.

```
// debug database  
debugImplementation  
  
"com.amitshekhar.android:debug-db:1.0.5"  
"
```

# **Shared Preferences**

*(Key Value Data)*



# Shared Preferences(Key-Value data)

Una de las herramientas más sencillas pero ampliamente utilizadas en Android para persistencia son las [Shared Preferences](#).

Esta nos permite **almacenar información sencilla** en un formato clave-valor. Los tipos de datos que podemos almacenar son datos primitivos, es decir *String*, *Integer*, *Boolean*, *Long* y *Sets de Strings*.

Cuando almacenamos datos en **Shared Preferences**, se crea un archivo al cual solo tiene acceso nuestra app si así lo indicamos. Podremos manejar la forma en que se crean y se borran estos registros.

No debemos confundir `SharedPreferences` con `Preferences`, esta última se utiliza para construir pantallas de preferencias y estos datos son almacenados en otros archivo de `Shared Preferences`.



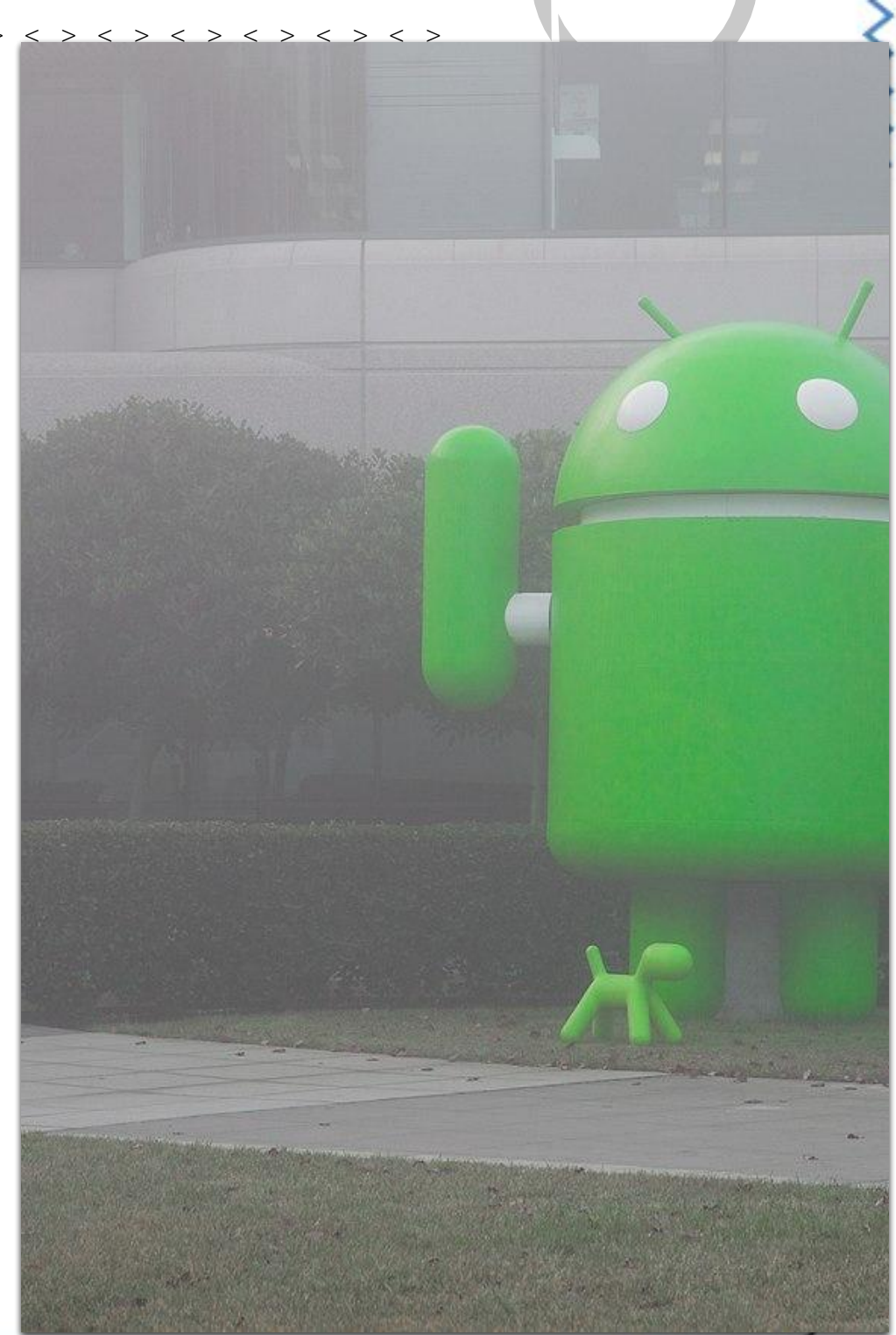




# Shared Preferences

## Ventajas:

- implicidad: implementación sencilla y rápida.
- Ligereza: adecuadas para datos pequeños como preferencias o configuraciones.
- Eficiencia: acceso rápido a los datos sin necesidad de consultas complejas.
- Privacidad: los datos sólo son accesibles por la aplicación que los creó.





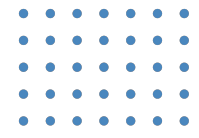


# Shared Preferences

## Desventajas:

- Limitación en tipos de datos: adecuadas sólo para datos simples.
- No estructurados: no son adecuadas para datos con relaciones complejas.
- Capacidad limitada: pueden volverse ineficientes para almacenar grandes cantidades de datos.





# ¿Cómo escribir en las shared Preferences?



- Para poder escribir en un archivo de sharedPreferences necesitamos utilizar la función **edit()**. Se le conoce como editor.
- Para que se apliquen los cambios podemos utilizar el método **apply()** lo cual escribiría los cambios de forma asíncrona.
- Alternativamente podemos usar **commit()** lo cual escribirá de forma síncrona. Tratar de no usarla con actividades muy pesadas para no bloquear la UI.

```
var mUserKey = "user"  
var mUser = "Mariocanedo2015@gmail.com"  
mSharedPreferences.edit().putString(mUserKey, mUser).apply()
```



# ¿Cómo escribir en las shared Preferences?

```
mSharedPreferences.edit().putString("TestString", "Hola").apply()
mSharedPreferences.edit().putBoolean("TestBoolean", true).apply()
mSharedPreferences.edit().putFloat("TestFloat", 1.0f).apply()
mSharedPreferences.edit().putInt("TestInt", 1).apply()
mSharedPreferences.edit().putLong("TestLong", 1L).apply()

val setString = setOf("one", "two", "three", "four")
mSharedPreferences.edit().putStringSet("TestStringSet", setString).apply()
```

# Cómo utilizar las Shared Preferences

- Para crear un archivo Shared Preferences en una app, en modo privado, debemos ejecutar el código de ejemplo, en general necesitaremos un context para utilizarlas.
- Existen otros modos para inicializarlas, pero están deprecados.

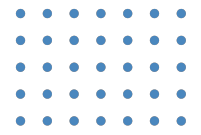
```
// Constante para el nombre del
archivo sharedPreferences.
private const val mFileNameSharedPreferences =
    "shared_preferences.xml"

class MainActivity : AppCompatActivity() {

    // Tener una variable de referencia para
    Shared Preferences
    lateinit var mSharedPreferences :
    SharedPreferences

{ override fun onCreate(savedInstanceState:
    Bundle?)

        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        //3. Instanciar las shared Preferences.
        mSharedPreferences = getSharedPreferences(
            mFileNameSharedPreferences,
            Context.MODE_PRIVATE)
    }
}
```



# Cómo leer las shared Preferences

Para poder leer desde un archivo de SharedPreferences debemos usar lo siguiente:

```
// Para leer de las shared Preferences var mUserRead =  
mSharedPreferences.getString(mUserKey, "")  
Log.d("TAG", mUserRead )  
...  
  
//para saber si contiene cierta clave  
mSharedPreferences.contains("TestString")  
//si quiero todos los valores y claves contenidos  
//pasa de ser getAll() a all gracias a kotlin mSharedPreferences.all  
//obtener un boolean  
mSharedPreferences.getBoolean("TestBoolean", false)  
//obtener un int mSharedPreferences.getInt("TestInt", -1)  
//obtener un long  
mSharedPreferences.getLong("TestLong", -1L)  
//obtener un Float mSharedPreferences.getFloat("TestFloat", -1.0f)  
//obtener un String  
mSharedPreferences.getString("TestString", "NotFound")  
//obtener un Set<String> mSharedPreferences.getStringSet("TestStringSet", setOf())
```

En definitiva con la función getString, getLong, getInt ... podemos obtener el dato indicando la clave con la cual fue guardada.



Momento:

# Time-out!



5 -10 min.





# PRACTICANDO LO APRENDIDO



Tiempo 30 minutos

# Ejercicio (Shared Preferences) 01

- Crear una nueva app para utilizarla de base para Shared Preferences
- Utiliza el template **basic Activity**.
- Vamos a generar una app como a de la izquierda.

**TIP:** Añade View Binding o usar Kotlin synthetic.





# Ejercicio (Shared Preferences) 02

- Crear los elementos de vistas según la imagen.
- No olvides poner los id de los elementos.
- Crear una instancia de SharedPreferences y el nombre que le asignaremos al archivo.
- Crearemos una llave para cada valor que guardaremos.



# Ejercicio (Shared Preferences) 03

- En onCreateView() del FirstFragment instanciamos sharedPreferences utilizando el contexto, y el nombre que generamos.
- Generar una función (save) que nos permite leer desde la vista y almacenar en las sharedPreferences.
- Validar que los campos no esten vacios y luego se procede a guardar en SP.
- Utilizamos:
  - putString()
  - putInt()
  - putBoolean()
  - putFloat()



# Ejercicio (Shared Preferences) 04

- Ahora vamos a leer los datos de las sharedPreferences y los pasaremos a las vistas.
- Crearemos un método para leer y mostrar los cambios.
- Los metodos claves son:
  - getInt
  - getString
  - getBoolean
  - getDecimal
- Una vez obtenidos los valores los pasamos a variables y luego a las vistas.



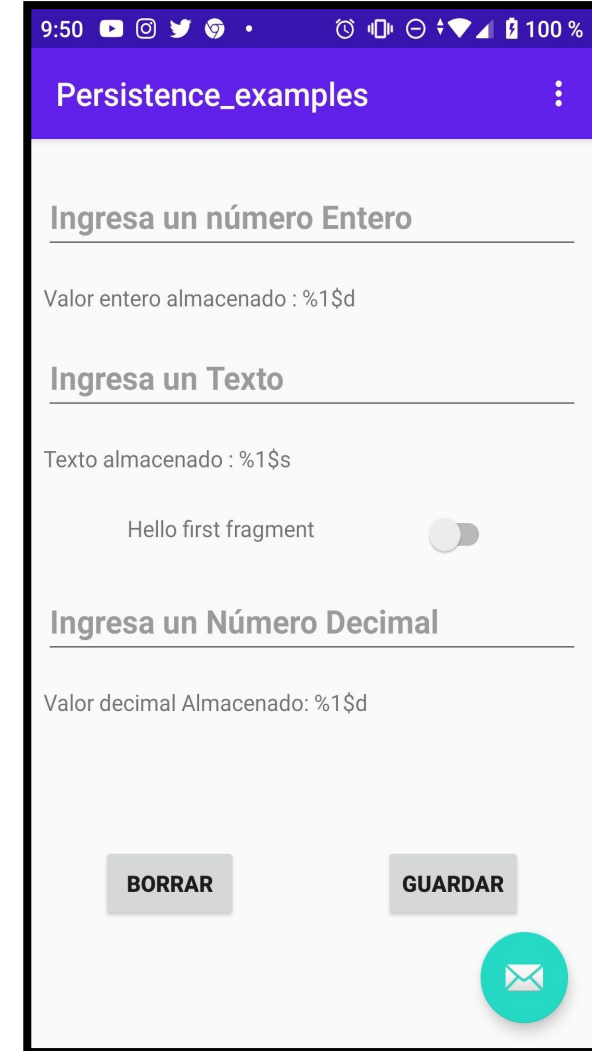
# Ejercicio (Shared Preferences) 05

- Creemos un método que borre todos los datos.
- Utilizar la función `.edit()`
- Esto nos permite utilizar el método `clear()` y aplicar el cambio con `apply()`;



# Ejercicio (Shared Preferences) 06

- En `onCreateView` utilizaremos el método para leer y llenar las vistas.
- También asignaremos listeners a nuestros botones y llamaremos a los métodos en su interior.



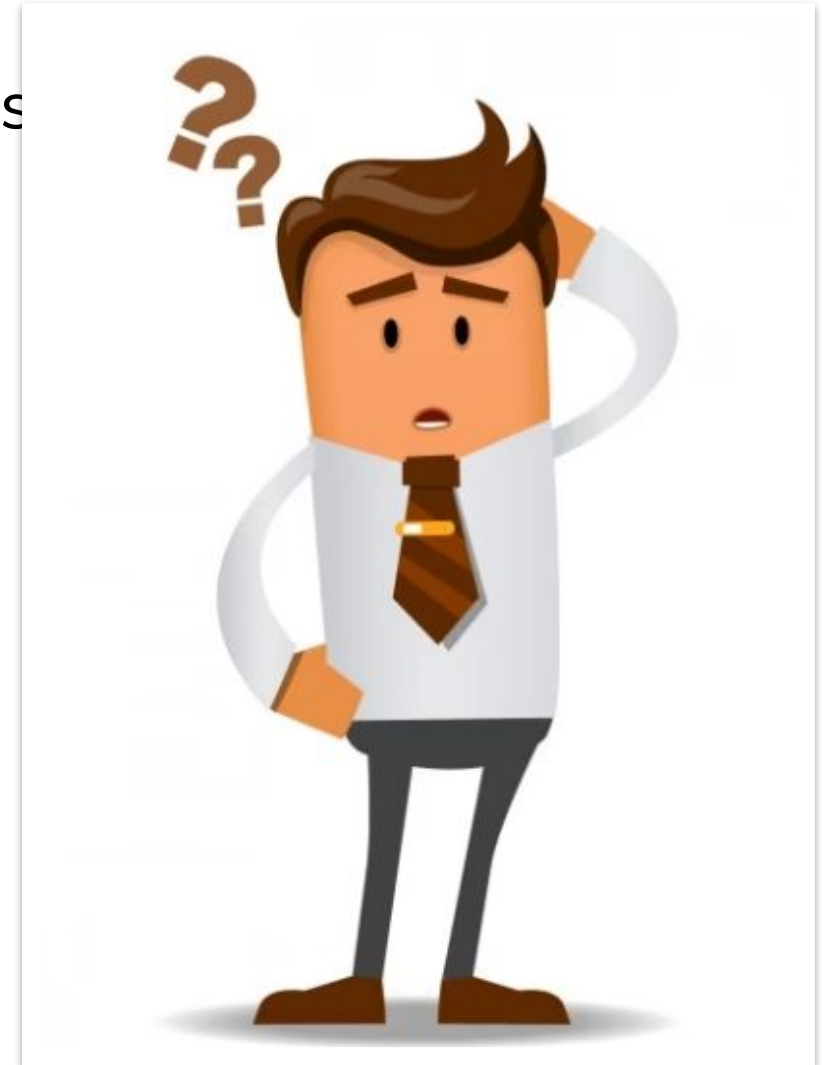


# Review del ejercicio y Dudas

Luego de este ejercicio, debemos tener más o menos claro cómo:

- Crear un archivo de SharedPreferences
- Guardar Datos
- Leer Datos
- Borrar Datos

¿Qué sucede si giras la pantalla?



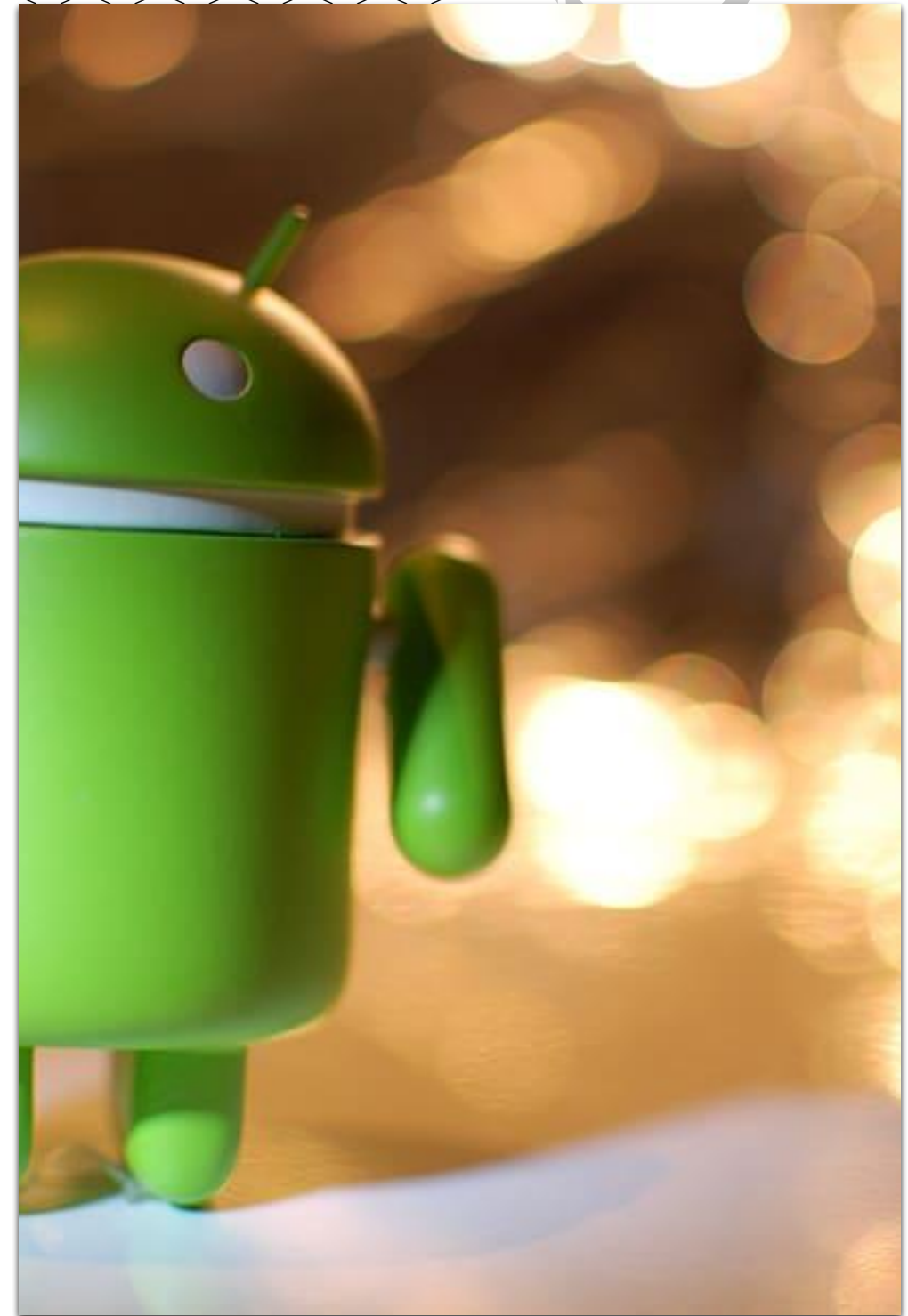
# Resumen y características de persistencia

|                       | Tipo de contenido                                   | Método de acceso | Necesidad de permisos                                                                                                                                              | Acceso desde otras apps                        | Se borran los archivos al desinstalar la app? |
|-----------------------|-----------------------------------------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|-----------------------------------------------|
| <a href="#">Media</a> | Shareable media files (images, audio files, videos) | MediaStore API   | <p>READ_EXTERNAL_STORAGE</p> <p>WRITE_EXTERNAL_STORAGE</p> <p>Los permisos son requeridos para todas las versiones desde Android 9 (API level 28) hacia abajo.</p> | Si, a través del permiso READ_EXTERNAL_STORAGE | No                                            |

# Roles de Shared Preferences

Almacenamiento de configuración:

Las SharedPreferences son ampliamente utilizadas para almacenar configuraciones de aplicaciones, como preferencias de usuario, ajustes de notificaciones y configuraciones generales. Esto permite que los usuarios personalicen la experiencia de la aplicación de acuerdo a sus preferencias.





# Roles de Shared Preferences

Mantenimiento del estado de la aplicación:

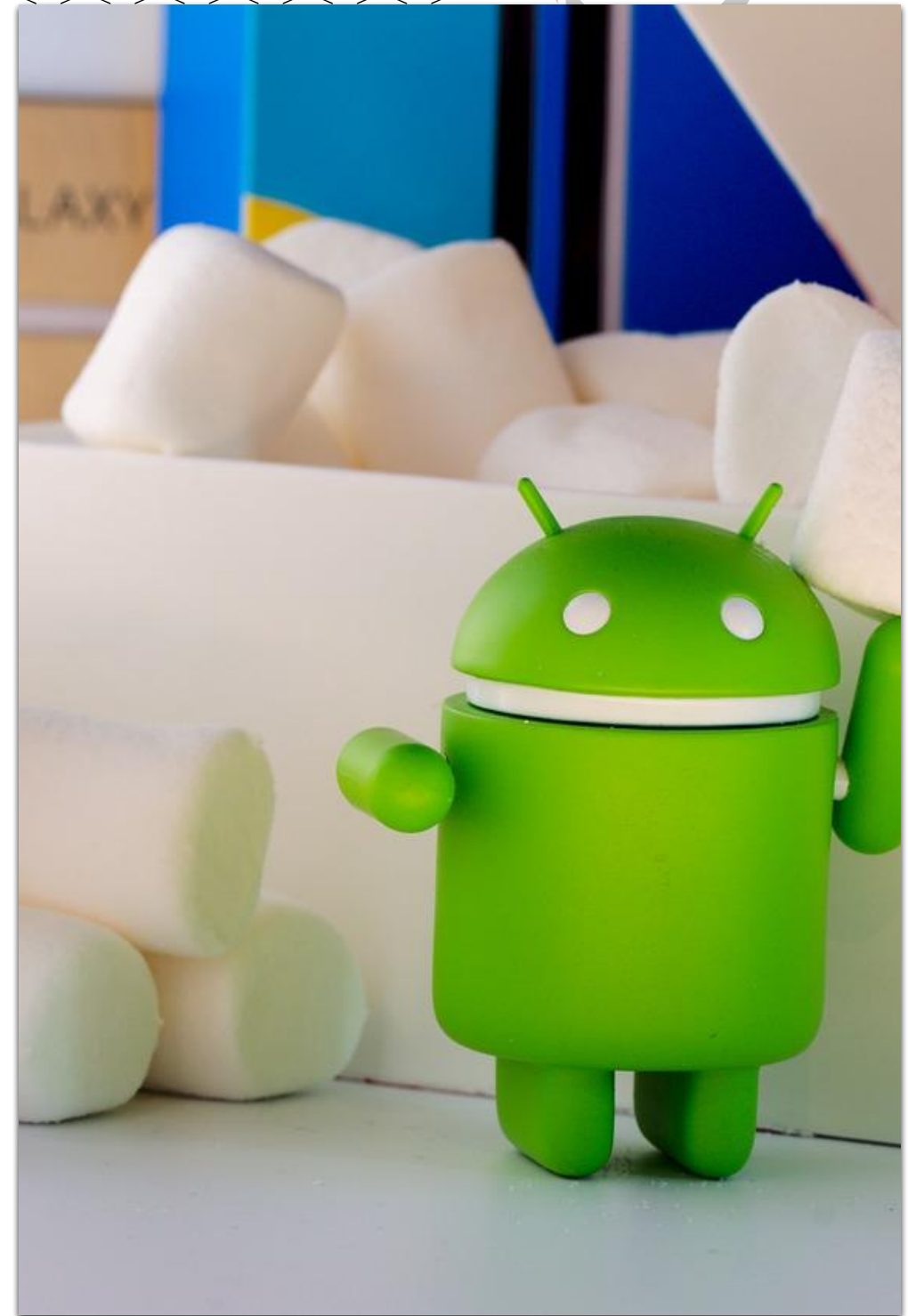
Puedes utilizar las SharedPreferences para almacenar y recuperar el estado de la aplicación. Por ejemplo, puedes guardar el estado de un juego o la última ubicación conocida del usuario.



# Roles de Shared Preferences

Control de sesiones y autenticación:

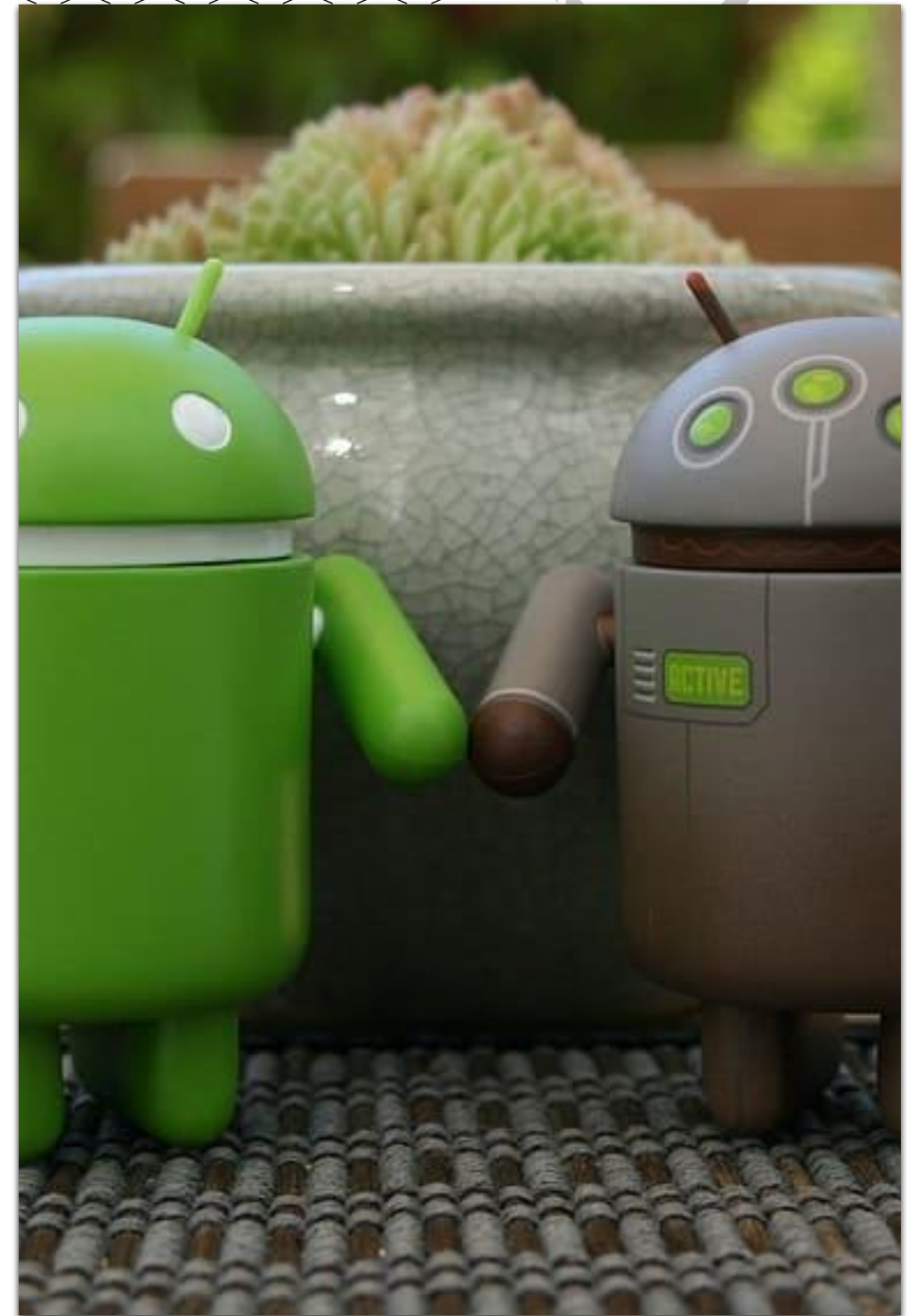
Las SharedPreferences a menudo se utilizan para almacenar tokens de autenticación, identificadores de sesión y otros datos relacionados con la seguridad. Esto permite a los usuarios mantenerse autenticados en la aplicación incluso después de cerrarla y volver a abrirla.



# Roles de Shared Preferences

Recopilación de información inicial:

Puedes utilizar las SharedPreferences para recopilar información inicial de los usuarios, como tutoriales vistos o configuraciones iniciales, y recordar estas elecciones en futuras sesiones.



# Roles de Shared Preferences

Personalización de la interfaz de usuario:

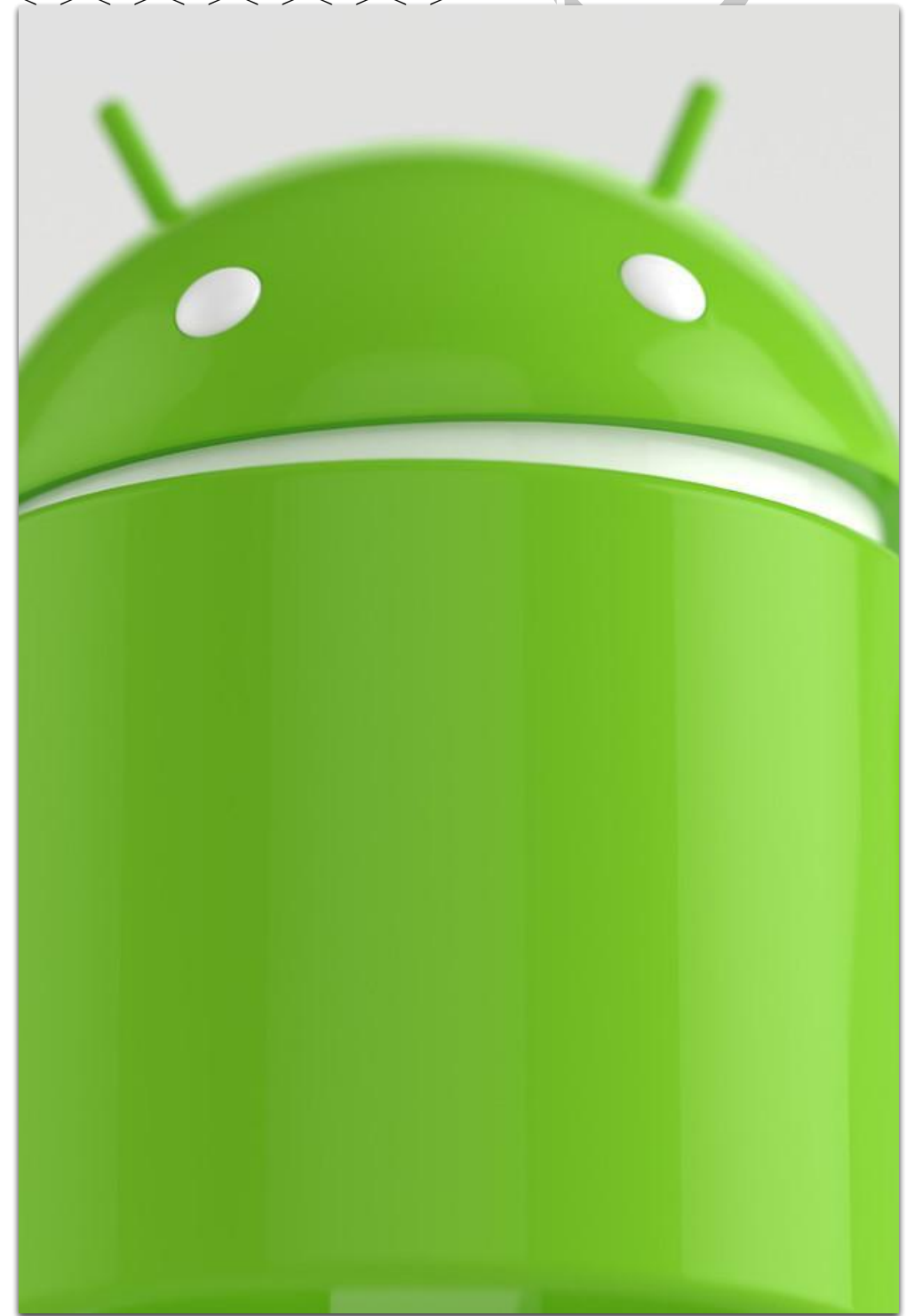
Puedes utilizar las SharedPreferences para adaptar la interfaz de usuario de la aplicación según las preferencias del usuario, como el tema, el idioma o la fuente predeterminada.



# Roles de Shared Preferences

Almacenamiento de datos pequeños

Las SharedPreferences son adecuadas para almacenar datos pequeños y simples, como números enteros, cadenas de texto, valores booleanos, flotantes, etc. No son adecuadas para almacenar grandes cantidades de datos o datos estructurados complejos.



# ACTIVIDAD

## **SharedPreferences:**

*Vamos a crear una simple clase que se encargue de guardar los datos de un usuario ( Email y Contraseña)*

*La idea es que puede implementar El patrón de arquitectura MVVM*

*y Preferencias.*

**Tiempo: 30 min**



# SOLUCIÓN

[https://github.com/Mariocanedo/SharedPreferences-MVVM/blob/master/app/src/main/java/com/example/sharedpreferences\\_v\\_14\\_05/MyModel.kt](https://github.com/Mariocanedo/SharedPreferences-MVVM/blob/master/app/src/main/java/com/example/sharedpreferences_v_14_05/MyModel.kt)



**Tiempo: 30 min**

○

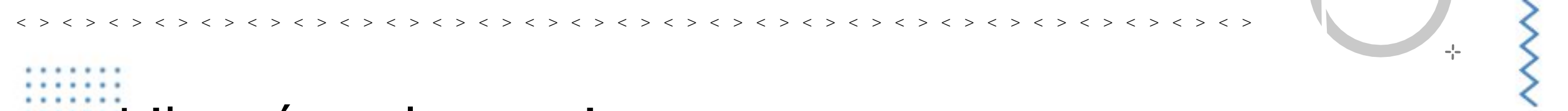
# ¿Alguna consulta?

+





# › Bases de datos en Android



# Librerías alternativas y SQLite

¿Por qué?:

Cuando trabajamos con datos hay muchas alternativas para almacenarlos dependiendo de la complejidad del proyecto, la cantidad de datos y la velocidad a la que necesitemos almacenarlos y recuperarlos.

Diferentes soluciones tienen respuestas a diferentes necesidades y tenemos que conocer las diferentes formas de llamar bases de datos y otros métodos de almacenamiento



# Bases de datos

# ¿Qué es una base de datos?

Una base de datos es un conjunto organizado de datos relacionados y estructurados que se almacenan de manera que puedan ser fácilmente accedidos, administrados y actualizados. Cuando trabajamos en bases de datos en Android usamos la implementación denominada SQLite, una versión más ligera y simple de SQL.





9:41

# Registrarse y empezar a transferir

Nombre

Ingresar nombre

Apellido

Ingresar apellido

Email

Ingresar Email

Contraseña

Ingresar contraseña

Reingresar contraseña

Reingresar contraseña

Crear Cuenta

Ya tienes una cuenta??



Servidor: localhost » Base de datos: alkemy » Tabla: users

Examinar Estructura SQL Buscar Insertar

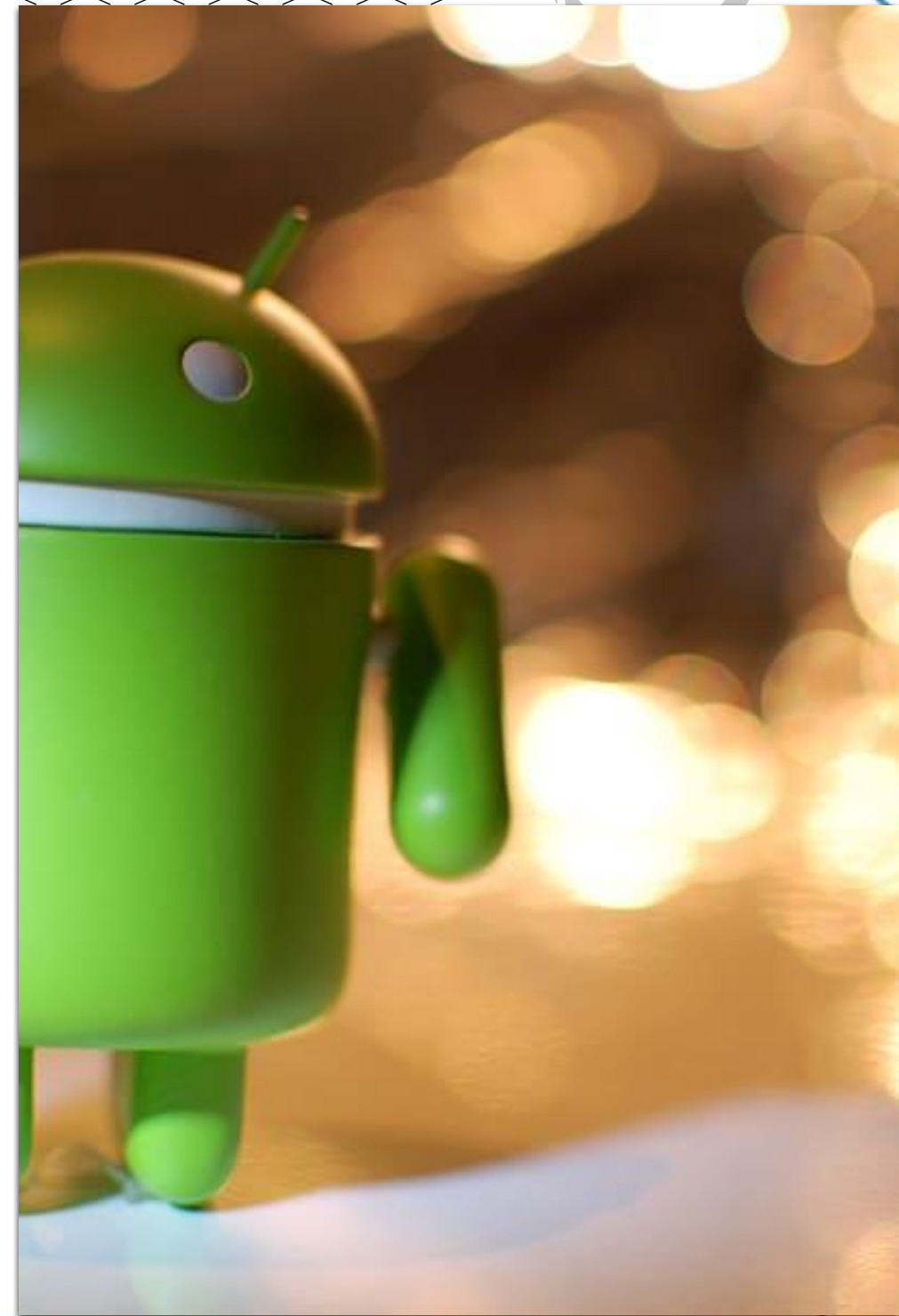
Estructura de tabla Vista de relaciones

|                          | # | Nombre           | Tipo        | Cotejamiento       | Atributos | Nulo | Predeterminado |
|--------------------------|---|------------------|-------------|--------------------|-----------|------|----------------|
| <input type="checkbox"/> | 1 | <b>user_id</b>   | int(11)     |                    |           | No   | Ninguna        |
| <input type="checkbox"/> | 2 | <b>name</b>      | varchar(50) | utf8mb4_general_ci |           | No   | Ninguna        |
| <input type="checkbox"/> | 3 | <b>last_name</b> | varchar(50) | utf8mb4_general_ci |           | No   | Ninguna        |
| <input type="checkbox"/> | 4 | <b>email</b>     | varchar(50) | utf8mb4_general_ci |           | No   | Ninguna        |
| <input type="checkbox"/> | 5 | <b>password</b>  | varchar(50) | utf8mb4_general_ci |           | No   | Ninguna        |
| <input type="checkbox"/> | 6 | <b>enabled</b>   | tinyint(1)  |                    |           | No   | Ninguna        |

# SQLite

**Sin servidor:** A diferencia de sistemas de gestión de bases de datos más complejos, SQLite es una biblioteca que se vincula directamente con la aplicación. No requiere un servidor de base de datos independiente.

**Autónomo:** SQLite es autónomo y no depende de un proceso separado en el sistema. Esto facilita su integración en aplicaciones y sistemas embebidos.



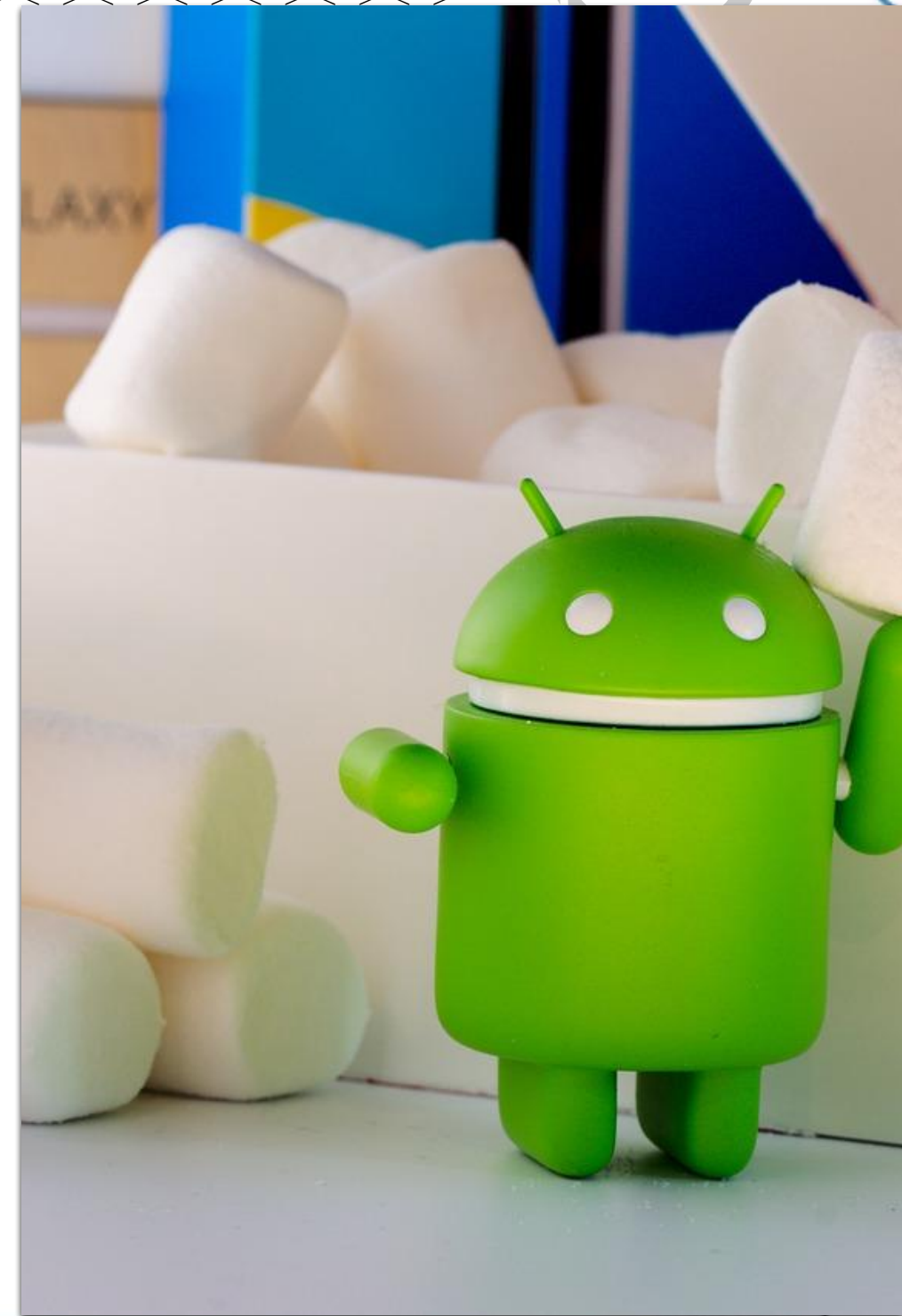


# SQLite

**Sin configuración:** No es necesario configurar SQLite antes de usarlo. Puedes comenzar a trabajar con bases de datos SQLite en una aplicación tan pronto como la incluyas.

**Sin acceso en red:** SQLite no se comunica a través de una red, lo que lo hace adecuado para aplicaciones móviles o sistemas embebidos que no siempre tienen conectividad a la red.

**Transaccional:** SQLite admite transacciones ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) para garantizar que las operaciones en la base de datos sean seguras y confiables.



# SQLite

Tipos de datos limitados: SQLite tiene un conjunto limitado de tipos de datos, que incluye INTEGER, REAL, TEXT y BLOB. Esto es más simple en comparación con algunos sistemas de bases de datos más grandes que ofrecen una amplia variedad de tipos de datos, como DATE, TIME, BOOLEAN, etc. Sin embargo, puedes almacenar una variedad de datos en estos tipos básicos.





# ➤ ORM

# ORM

¿Qué es?:

Un ORM, o Mapeador Objeto-Relacional (Object-Relational Mapping en inglés), es una técnica de programación que permite a las aplicaciones interactuar con bases de datos relacionales utilizando objetos y modelos de datos en lugar de sentencias SQL directas. Se encarga de asignar objetos y clases de la aplicación a tablas y columnas en la base de datos relacional. También gestiona la transformación de datos entre la representación de objetos en el código de la aplicación y la representación relacional en la base de datos.

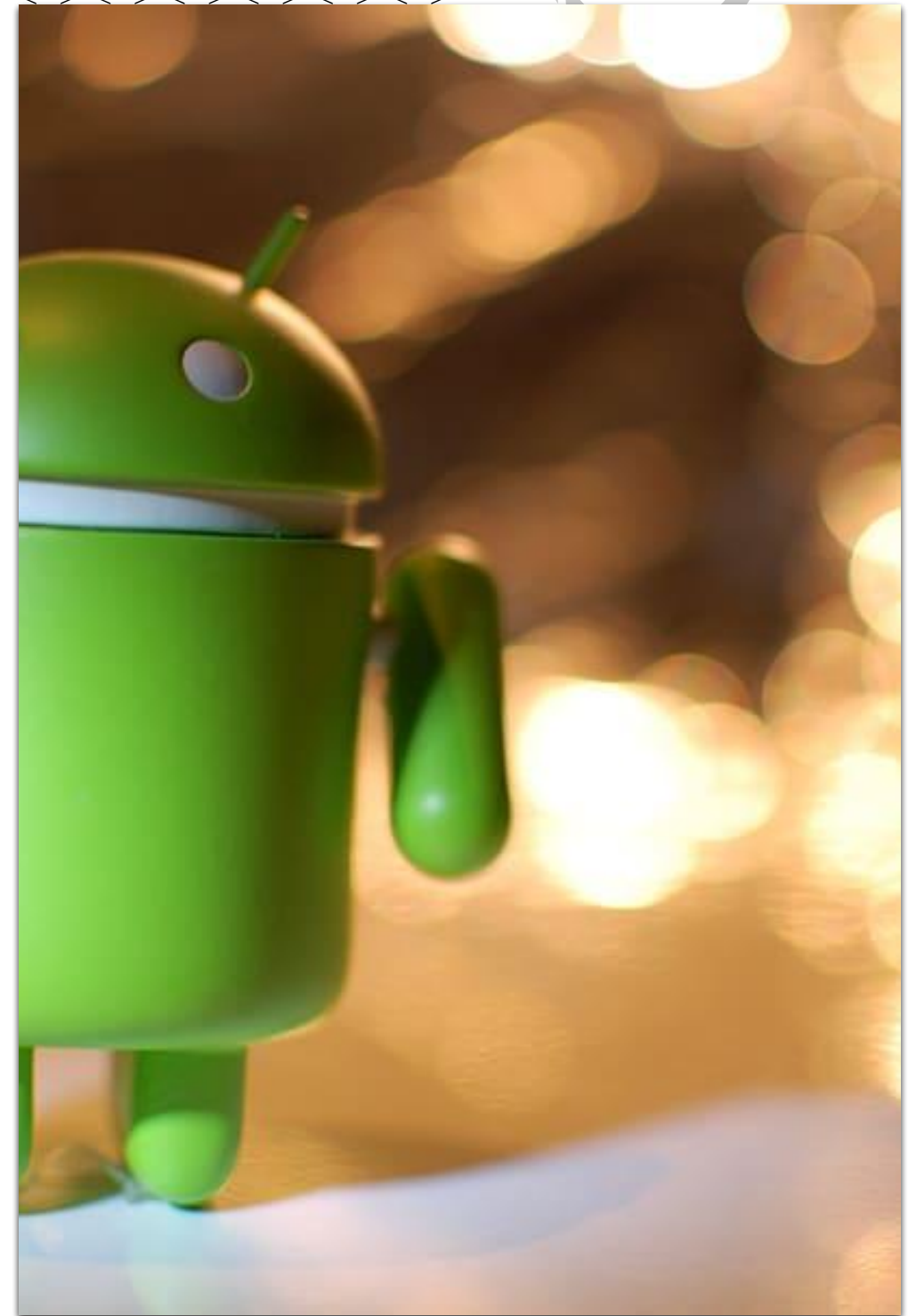
# DAO

¿Qué es?:

Un DAO es una clase o componente que proporciona métodos para acceder y manipular datos en una base de datos. Su responsabilidad es encapsular todas las operaciones de consulta, inserción, actualización y eliminación de datos relacionadas con una entidad o tabla específica en la base de datos.

# Ventajas de usar ORM

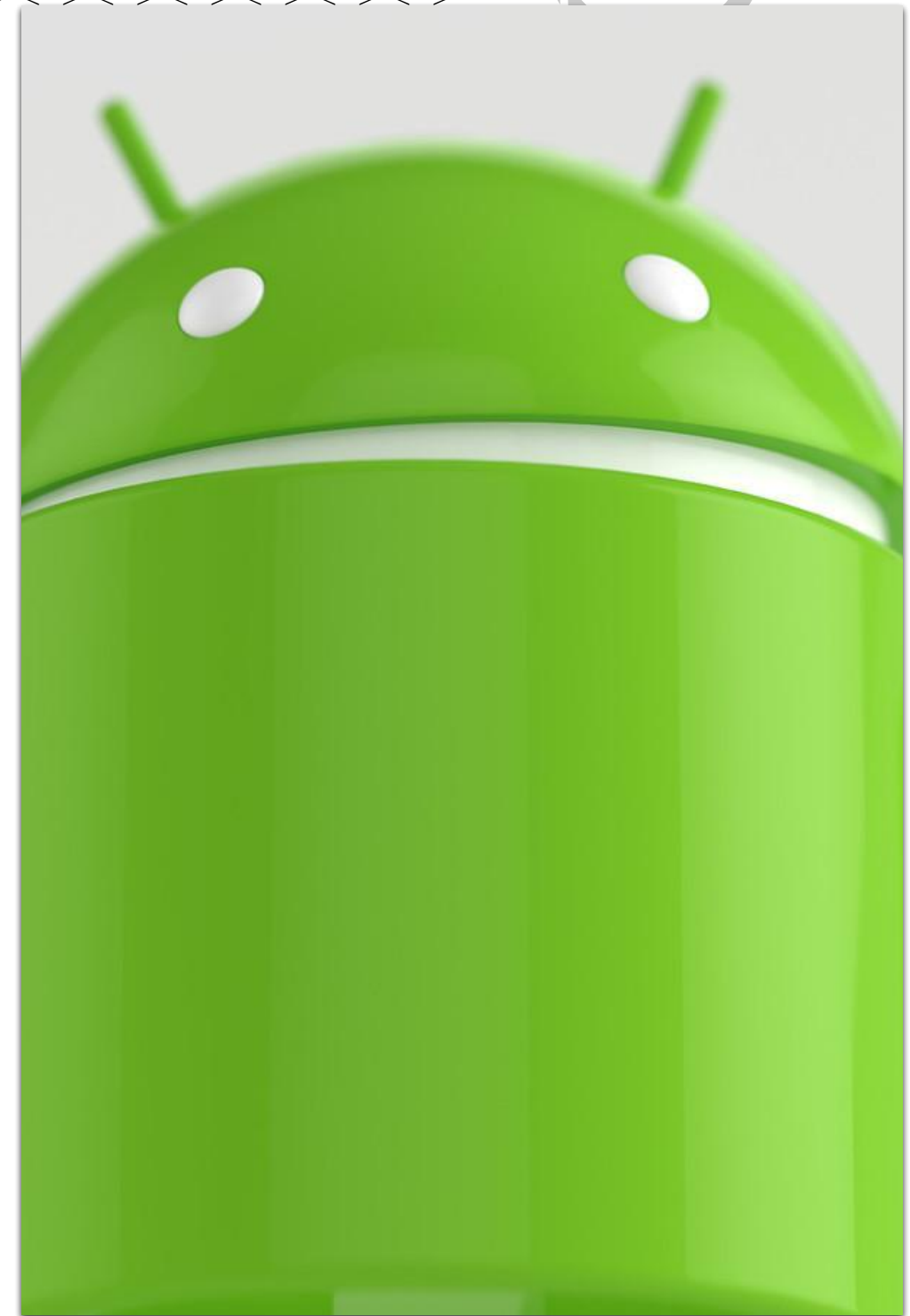
Abstracción de la Base de Datos: Los ORMs proporcionan una capa de abstracción entre la lógica de la aplicación y la base de datos subyacente. Esto significa que los desarrolladores pueden trabajar con objetos y clases en lugar de escribir SQL directamente. La abstracción de la base de datos facilita la programación al ocultar los detalles específicos de la base de datos y permite un enfoque más orientado a objetos en el desarrollo de aplicaciones.



# Ventajas de usar ORM

Mayor Productividad:

Al eliminar la necesidad de escribir SQL manualmente para operaciones de acceso a datos, los desarrolladores pueden ser más productivos. Los ORMs generan automáticamente el SQL necesario para interactuar con la base de datos, lo que acelera el desarrollo y reduce la cantidad de código que se debe escribir y mantener.



# Ventajas de usar ORM

## Seguridad:

Los ORMs suelen incorporar medidas de seguridad, como la prevención de ataques de inyección SQL, al generar SQL de manera segura. Esto reduce la probabilidad de vulnerabilidades de seguridad en la aplicación.



# Ventajas de usar ORM

## Reutilización de Código:

Los ORMs promueven la reutilización de código al proporcionar operaciones de acceso a datos comunes que se pueden utilizar en múltiples partes de la aplicación. Esto reduce la duplicación de código y facilita el mantenimiento.



# Ventajas de usar ORM

Optimización de Consultas:

Algunos ORMs ofrecen herramientas para optimizar las consultas generadas, lo que puede mejorar el rendimiento de la aplicación y reducir la carga en la base de datos.

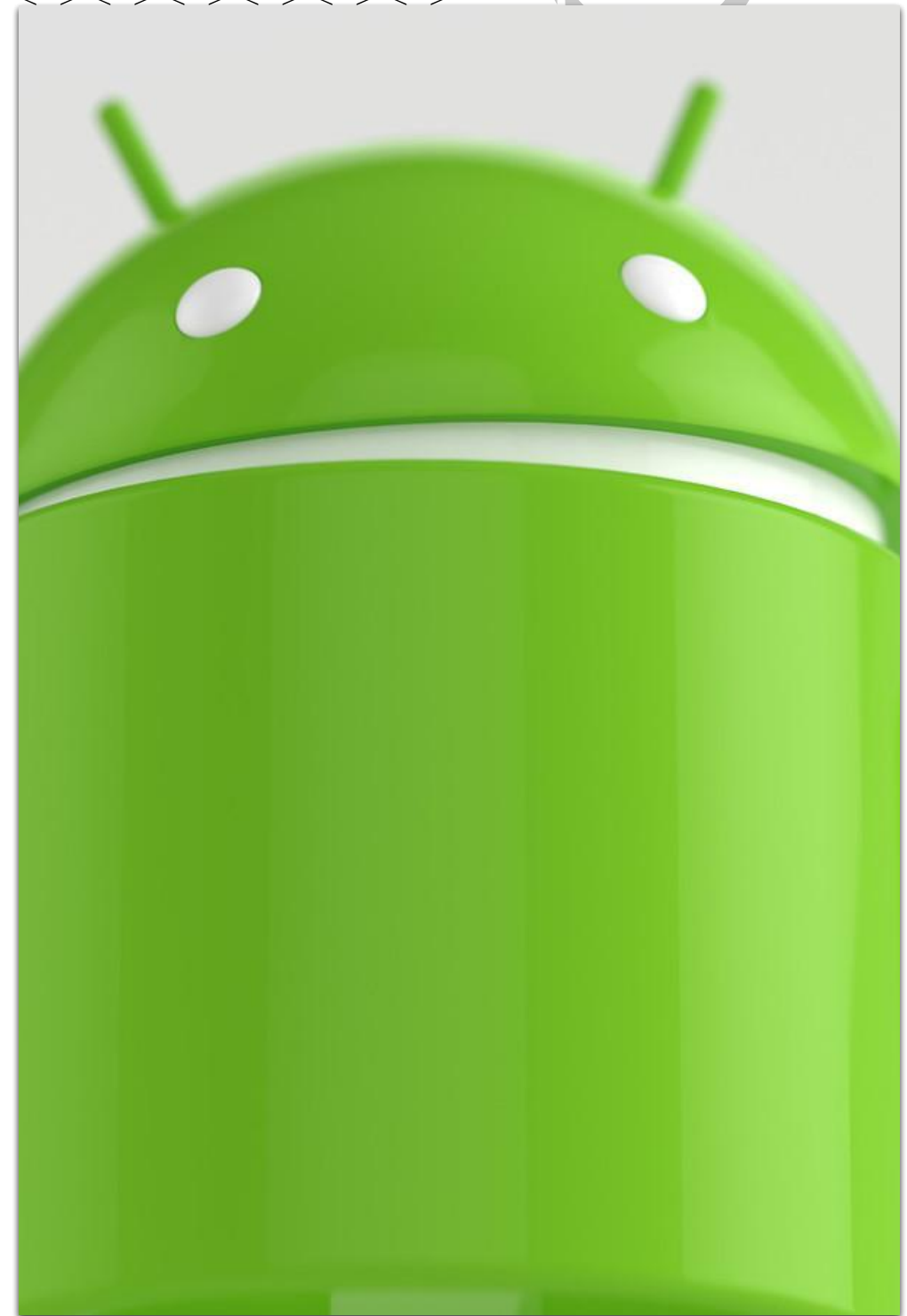




# Librerías

Realm:

Es una base de datos orientada a objetos para dispositivos móviles. Proporciona una alternativa más rápida y fácil de usar a las bases de datos tradicionales en Android. Realm permite el almacenamiento de objetos directamente en la base de datos sin necesidad de un mapeo explícito entre objetos y tablas. También ofrece características como consultas en tiempo real, transacciones en bloque y sincronización en tiempo real. Realm es conocido por su rendimiento y su facilidad de uso.



# Librerías

## SQLDelight:

Es una librería de generación de código que permite trabajar con consultas SQL de manera segura y eficiente en aplicaciones Android. Con SQLDelight, se define el esquema de la base de datos y las consultas SQL en un archivo específico, y luego se genera el código Java o Kotlin correspondiente. Esto permite una verificación en tiempo de compilación de las consultas y una mejor integración con el código de la aplicación. SQLDelight se basa en el uso de SQLite como motor de base de datos y proporciona una forma robusta y eficiente de trabajar con bases de datos relacionales.



# Librerías

## GreenDao:

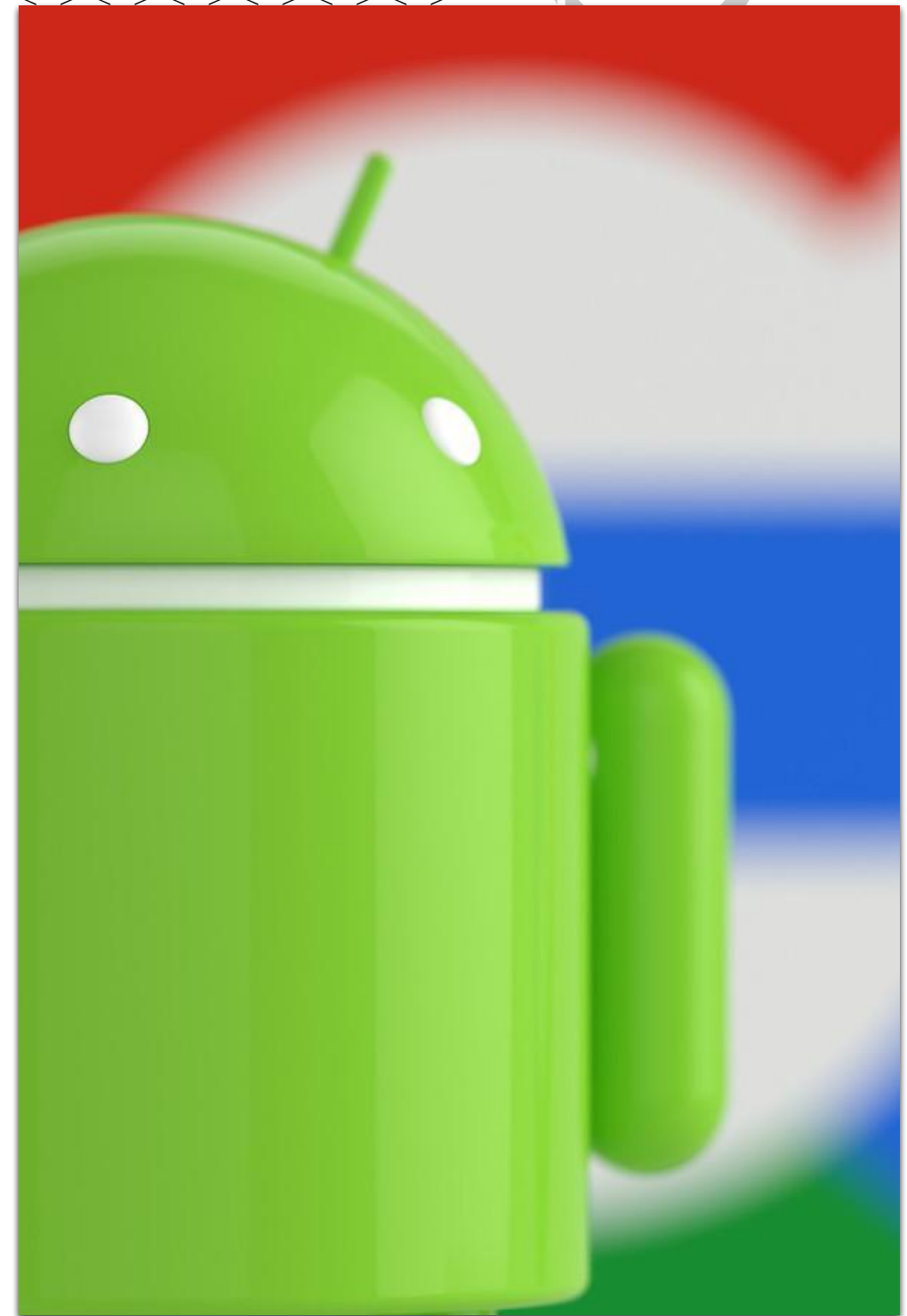
Es un generador de código de base de datos ORM (Object-Relational Mapping) ligero y eficiente para Android. Permite el mapeo de objetos Java a tablas de base de datos SQLite de manera sencilla. GreenDao genera el código necesario para realizar operaciones de inserción, actualización, eliminación y consulta de datos de manera eficiente. También ofrece características como relaciones de objetos y soporte para transacciones. GreenDao se destaca por su rendimiento y su enfoque ligero.



# Librerías

Room:

Room es una biblioteca en Android que facilita la creación, administración y acceso a bases de datos SQLite en aplicaciones Android. Esta biblioteca forma parte de la arquitectura de componentes de Android y se utiliza comúnmente junto con otras partes de la arquitectura de componentes, como LiveData y ViewModel, para crear aplicaciones Android robustas y eficientes.





¿Alguna consulta?





# Resumen

¿Qué logramos en esta clase?

- ✓ **SharedPreferences**
- ✓ **entajas**
- ✓ **Guardado de preferencias**
- ✓ **SQLite**
- ✓ **Características de base de datos en Android**
- ✓ **ORM**
- ✓ **DAO**

< **¡Muchas gracias!** >

